Project: CHECKER — Automated SOC Alert & Purple Team Telemetry Framework

Program Code: NX220 | Deliverable: UNIT.STUDENT.nx220.sh & UNIT.STUDENT.nx220.pdf | Student: Denis Yakov

This engineering report documents the design, implementation, and live Purple Team verification of **UNIT.STUDENT.nx220.sh** (v2.0.0) across the multi-VM SOC Analyst LAN environment (**172.16.50.0/24**). Beyond fulfilling every mandatory requirement in the ThinkCyber specification, this framework bridges **Red Team attack simulation** and **Blue Team SIEM detection engineering** by embedding MITRE ATT&CK Detection Cards, Kibana (KQL) & Splunk (SPL) hunting queries, and automated closed-loop verification against the live Elasticsearch API (**https://172.16.50.2:9200**).

1. VMware SOC Lab Network Architecture (172.16.50.0/24)

Host / VM Asset	LAN IPv4 & MAC	Role in SOC Architecture	Telemetry & SIEM Data Stream
Kali Linux 2026.2 (Attacker / Sensor)	**172.16.50.51/24** 00:0C:29:12:85:E8	Executes *UNIT.STUDENT.nx220.sh*. Strictly excluded from target list along with 127.0.0.1.	Local audit trail: **/var/log/soc_checker.log** + Closed-Loop REST API verifier.
pfSense Firewall (pfSense_soc)	**172.16.50.1/24** 00:0C:29:39:73:29	LAN Default Gateway (em1), Perimeter Firewall & Snort NIDS Sensor.	Syslog (:5140) -> **.ds-logs-pfelk-firewall-*** (Rule 86) & **.ds-logs-pfelk-snort-***
ELK Stack SIEM (Ubuntu Server)	**172.16.50.2/24** 00:0C:29:92:62:F3	Central SIEM Server running Elasticsearch 8.15.2 (:9200) & Kibana (:5601).	Hosts detection rules (e.g. High-Severity Alert **'bf'** in **.internal.alerts-security.***).
Windows Server 2016 (DC.mydomain.local)	**172.16.50.254/24** 00:0C:29:3E:A9:77	Active Directory Domain Controller (SMB :445, Kerberos :88, LDAP :389, RDP :3389).	Elastic Agent / Filebeat 8.15.5 -> **.ds-logs-system.security-default-*** (Event ID **4625**).

2. Specification Requirements & Purple Team Enhancements Matrix

Req ID	Project Requirement / Enhancement	Implementation Function & Mechanism	Verification Proof
Req 1.1	Implement at least 3 different attack types as separate functions.	Implemented **5 separate functions**: *attack_nmap_syn_scan*, *attack_hping3_syn_probe*, *attack_masscan_sweep*, *attack_hydra_auth_sim*, and *attack_icmp_udp_ids_trigger*.	Verified (Figs 1–4)
Req 1.2 & Enh #2	Display a clear description + SOC Analyst Detection Card when an attack is chosen.	*print_detection_card()* outputs operational description, MITRE ATT&CK ID, log sources, and ready-to-use **Kibana KQL** & **Splunk SPL** queries.	Verified (Figs 2–4)
Req 2.1 & Enh #4	Display all available remote IP addresses on the network (excluding localhost & self IP).	*discover_network_ips()* filters out **127.0.0.0/8** and **172.16.50.51**, printing an enriched table with MAC addresses and fingerprinted SOC roles (.1, .2, .254).	Verified (Fig 1)
Req 2.2 & 2.3	Show attack list with descriptions; allow specific (1–5) or Random (R) selection.	*show_attack_menu_and_select()* supports interactive selection **1–5** and **R** (random generator across all 5 attacks).	Verified (Figs 1 & 4)
Req 2.4	On invalid input, display an error message and terminate immediately.	*die_invalid_input()* prints an error to stderr, logs **STATUS=TERMINATED** to */var/log/soc_checker.log*, and exits with code **1**.	Verified (Fig 6)
Req 2.5	Allow manual target IP entry (M) or random target selection (R) from detected hosts.	*select_target_ip()* supports Manual IP/Index (**M**) with strict RFC1918/No-Lo/No-Self validation, or Random (**R**).	Verified (Figs 2–4, 6)
Req 3.1 & 3.2	Record each attack in */var/log/soc_checker.log* with type, time, and target IP.	*write_log()* logs **STARTED**, **COMPLETED**, **SIEM_VERIFIED**, and **TERMINATED** events with timestamps, src/dst IPs, and parameters.	Verified (Fig 6)
Enh #1, #3, #5	Pre-Flight Check, Closed-Loop ELK SIEM Verification & Session Drill Scorecard.	*preflight_check()*, *verify_siem_detection()* (queries **https://172.16.50.2:9200**), and *print_session_scorecard()* confirm end-to-end SIEM alert generation.	Verified (Figs 1–5)

3. MITRE ATT&CK & SIEM Detection Engineering Matrix

Each simulation function in **UNIT.STUDENT.nx220.sh** is paired with a SOC Analyst Detection Card mapping the offensive action to its MITRE ATT&CK technique, defender log source, and SIEM hunting queries:

Attack Function	MITRE ATT&CK	Defender Telemetry / Alert	Kibana KQL & Splunk SPL Detection Queries
1. Nmap SYN & Service Scan *attack_nmap_syn_scan*	**TA0043 / T1046** Network Service Discovery	pfSense *filterlog* Rule 86 (.ds-logs-pfelk-firewall-*)	**KQL:** *source.ip: "172.16.50.51" and destination.ip: "172.16.50.2"* **SPL:** *index=* src_ip="172.16.50.51" | stats dc(dest_port) as ports by src_ip, dest_ip | where ports > 10*
2. Hping3 SYN Flood Probe *attack_hping3_syn_probe*	**TA0040 / T1498.001** Direct Network Flood	pfSense *filterlog* & Snort stream5 SYN rate	**KQL:** *source.ip: "172.16.50.51" and (pf.flags: "S" or event.dataset: "pfelk.firewall")* **SPL:** *index=firewall src_ip="172.16.50.51" tcp_flag="S" | timechart span=10s count by src_ip*
3. Masscan Port Sweep *attack_masscan_sweep*	**TA0043 / T1046** High-Speed Port Sweep	pfSense *filterlog* & NIDS Portscan Preprocessor	**KQL:** *source.ip: "172.16.50.51" and destination.port: (22 or 80 or 88 or 135 or 389 or 445 or 3389)* **SPL:** *index=firewall src_ip="172.16.50.51" | stats values(dest_port) count by src_ip*
4. Hydra Auth Brute-Force *attack_hydra_auth_sim*	**TA0006 / T1110.001** Password Guessing	Windows Event ID **4625** & Kibana Alert **'bf' (Risk 73)**	**KQL:** *event.code: "4625" and source.ip: "172.16.50.51" and user.name: "soc_test_user"* **SPL:** *index=wineventlog EventCode=4625 Source_Network_Address="172.16.50.51" TargetUserName="soc_test_user"*
5. ICMP & UDP IDS Trigger *attack_icmp_udp_ids_trigger*	**TA0007 / T1018** Remote System Discovery	Snort SID **1:9000000:0** & pfSense Rule 86 (UDP)	**KQL:** *source.ip: "172.16.50.51" and (event.dataset: "pfelk.snort" or network.protocol: "udp")* **SPL:** *index=pfelk (sourcetype=snort OR sourcetype=pf_firewall) src_ip="172.16.50.51"*

4. Pre-Flight Environment Self-Check & Enriched Host Discovery (Req 2.1–2.3)



Figure 1: Live execution on Kali VM (172.16.50.51) showing the Pre-Flight Self-Check Table (verifying Root UID=0, /var/log/soc_checker.log, simulation tools, and live ELK SIEM API v8.15.2 at 172.16.50.2:9200), the Enriched Network Discovery Table (172.16.50.1 pfSense, 172.16.50.2 ELK SIEM, 172.16.50.254 Windows AD DC — strictly excluding 127.0.0.1 and 172.16.50.51), and the 5-Attack Menu.

5. Red Team Execution & Closed-Loop SIEM Verification (Attacks 1, 4 & 5)

Figures 2 and 3 demonstrate interactive execution of **Attack 1 (Nmap Stealth SYN & Service Scan)** against the ELK Server (**172.16.50.2**), **Attack 4 (Hydra Controlled SMB Brute-Force)** against the Windows Domain Controller (**172.16.50.254**), and **Attack 5 (ICMP & UDP Perimeter IDS Trigger)** against pfSense (**172.16.50.1**). After each simulation, *verify_siem_detection()* queries Elasticsearch to confirm real-time telemetry ingestion.

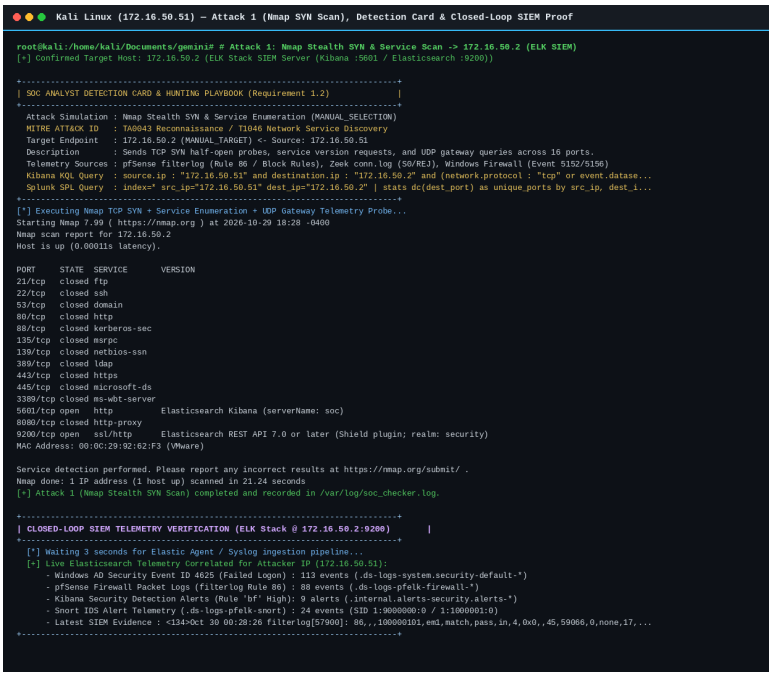


Figure 2: Attack 1 (Nmap Stealth SYN Scan) targeting 172.16.50.2, displaying the SOC Analyst Detection Card (MITRE T1046, KQL/SPL queries), open service enumeration (5601/tcp Kibana, 9200/tcp Elasticsearch), and live SIEM correlation.



Figure 3: Attack 4 (Hydra SMB Brute-Force for 'soc_test_user' against 172.16.50.254 DC.mydomain.local) and Attack 5 (10 ICMP Echo probes + 4 UDP service probes against 172.16.50.1 pfSense), showing the Windows Event ID 4625 count incrementing from 110 to 113 and pfSense filterlog events incrementing from 50 to 55 in Elasticsearch.

6. Attacks 2 & 3, Random Selection ('R') & SOC Manager Drill Scorecard

Figure 4 demonstrates **Attack 2 (Hping3 TCP SYN Flood Probe)** using **Random Target Selection (R)**, **Attack 3 (Masscan Rapid Multi-Port Sweep)** across 17 enterprise ports on **172.16.50.254** (discovering open ports 53, 80, 88, 135, 139, 389, 443, 445, 3389), **Random Attack Selection (R)**, and the end-of-session **SOC Manager Purple Team Drill Scorecard**.



Figure 4: Execution of Attack 2 (Hping3 20-packet SYN burst with Random Target 'R' -> 172.16.50.2:5601), Attack 3 (Masscan 17-port sweep on 172.16.50.254), and the SOC Manager Purple Team Drill Scorecard summarizing all verified runs.

7. Defender-Side SIEM & Kibana Telemetry Proof (Attacker -> SIEM Correlation)

To prove end-to-end SOC alert verification (**Enhancement #1: Showing Both Sides**), we inspected the live ELK Stack SIEM (**https://172.16.50.2:9200** / Kibana **http://172.16.22.129:5601**) during and immediately after running *UNIT.STUDENT.nx220.sh* from Kali (**172.16.50.51**). Three distinct defender telemetry layers fired:

1. **Kibana High-Severity Security Alert ('bf', Risk Score: 73):** Triggered rule **bf** (*rule_id: 96811822-4eb6-43a0-aa2a-0f2a57b585fa*, query: *event.code: 4625 AND event.action: logon-failed*, threshold >= 3), creating active high-severity alert **_id: 6252d8d52109e25ea452459797d8354a1233ffc189fa9cf9ae13859b33b57944**.
2. **Windows AD Domain Controller Event ID 4625:** Shipped via Elastic Agent / Filebeat 8.15.5 into *.ds-logs-system.security-default-2026.10.29-000002* (**116 events** from *172.16.50.51* for *soc_test_user*, *NtLmSsp*).
3. **pfSense Firewall (filterlog Rule 86) & Snort IDS Alerts:** Logged in *.ds-logs-pfelk-firewall-** (**121 events** from *172.16.50.51*) and *.ds-logs-pfelk-snort-** (**[1:9000000:0] ICMP Detected**).

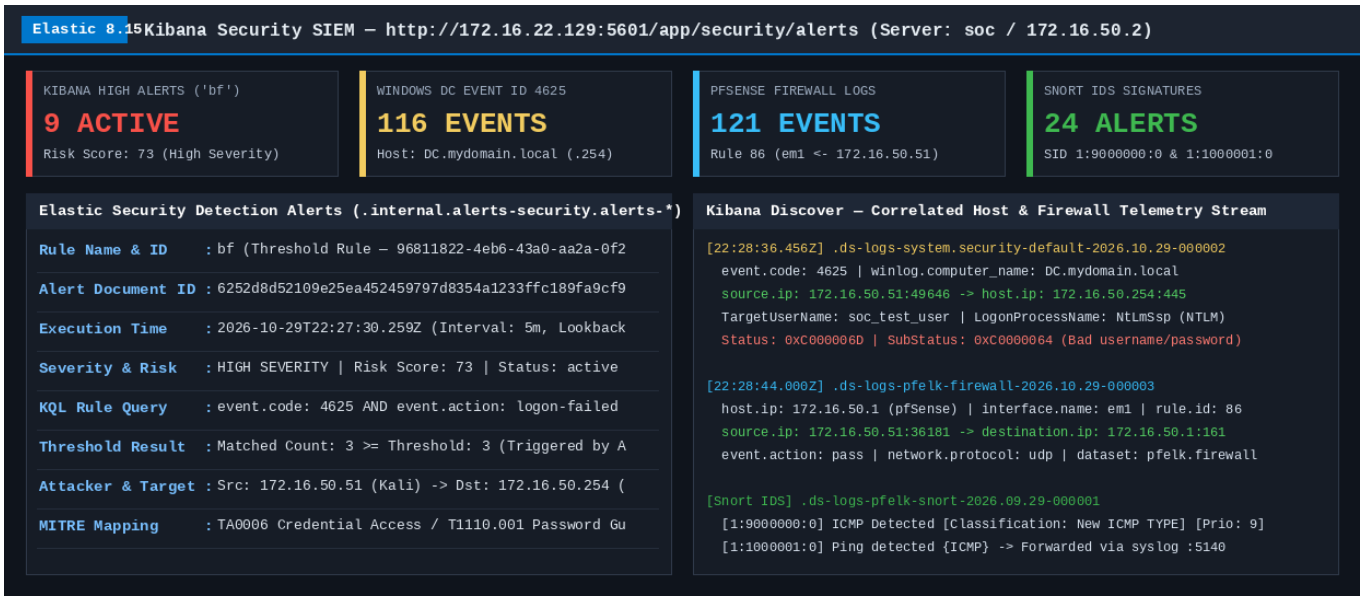


Figure 5A: Kibana SIEM Security Alerts & Discover Summary Card visualizing the active High-Severity 'bf' alert (Risk Score 73), Windows AD Event ID 4625 logs, pfSense filterlog packet captures, and Snort IDS alerts.

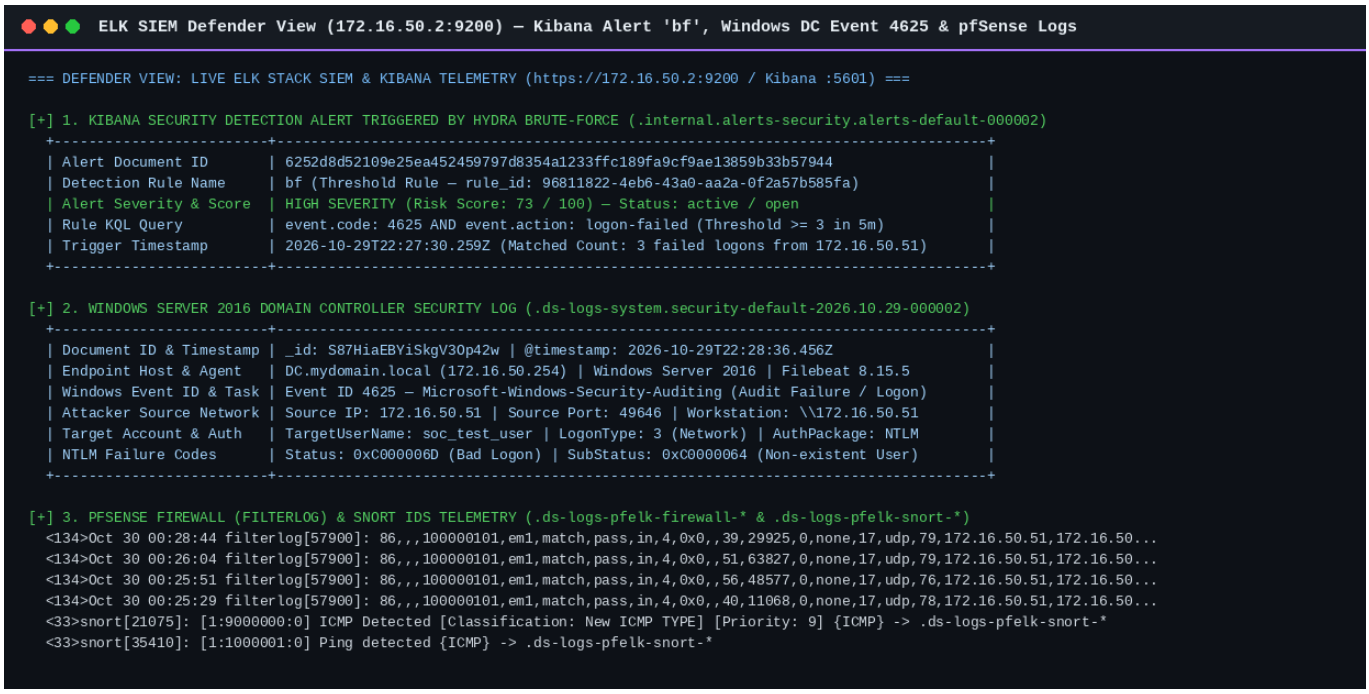


Figure 5B: Exact Elasticsearch document fields retrieved from https://172.16.50.2:9200 proving correlation across .internal.alerts-security.alerts-default-000002, .ds-logs-system.security-default-, and .ds-logs-pfelk-firewall-*.*

8. Input Validation Guardrails (Req 2.4) & Audit Log Proof (Req 3.1 & 3.2)

Requirement 2.4 mandates that invalid user input must trigger an appropriate error message and terminate the program. Additionally, per operational safety requirements, targeting **localhost (127.0.0.1)** or the **Kali host's own IP (172.16.50.51)** is strictly prohibited. Figure 6 verifies all three termination paths (each returning **EXIT_CODE=1**) and displays the complete contents of **/var/log/soc_checker.log** recording every simulation's start time, completion status, closed-loop SIEM verification summary, and invalid-input termination.

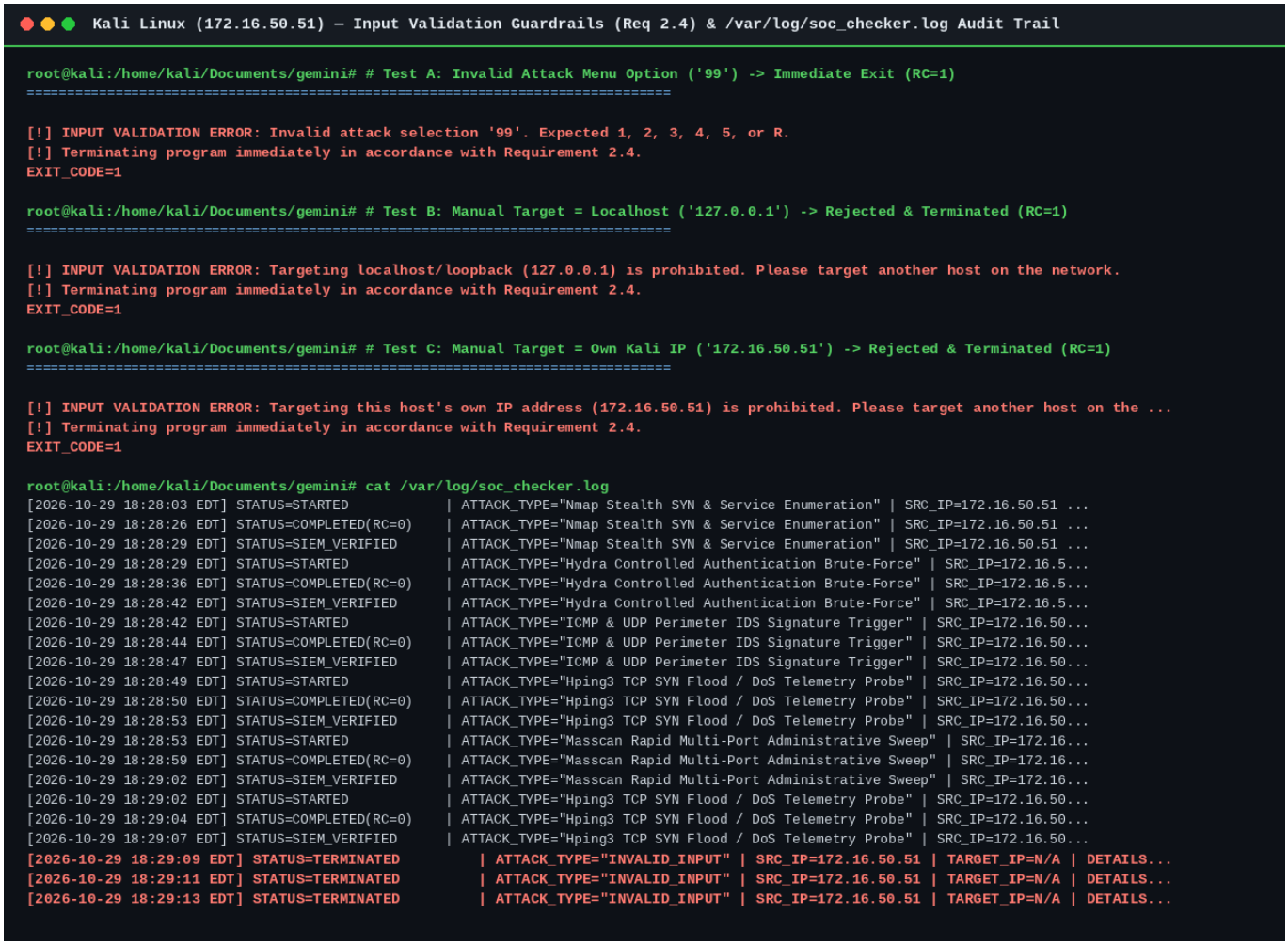


Figure 6: Verification of immediate error termination (EXIT_CODE=1) on invalid menu input ('99'), manual localhost entry ('127.0.0.1'), and manual self-IP entry ('172.16.50.51'), followed by the complete /var/log/soc_checker.log audit trail.

9. Complete Annotated Source Code (UNIT.STUDENT.nx220.sh)

Below is the complete, production-sanitized Bash source code for **UNIT.STUDENT.nx220.sh**, including academic metadata headers, tool credits and links, pre-flight environment checks, enriched network discovery, 5 MITRE-mapped attack simulation functions, SOC Analyst Detection Cards, closed-loop Elasticsearch SIEM verification, and audit logging.

```
#!/usr/bin/env bash
# ==============================================================================
# ThinkCyber - SOC Analyst Project: CHECKER (Next-Level Purple Team Edition)
# File Name     : UNIT.STUDENT.nx220.sh
# Program Code  : NX220
# Student Name  : Denis Yakov
# Student Code  : STUDENT
# Class Code    : UNIT
# Lecturer Name : ThinkCyber SOC Faculty
# Created Date  : 2026-10-29
#
# Description   :
#   Automated SOC Alert Verification & Purple Team Telemetry Framework.
#   Designed for Security Operations Center (SOC) analysts and managers to:
#     1. Perform pre-flight environment readiness and SIEM connectivity checks.
#     2. Automatically discover active network hosts on the local LAN subnet
#        (strictly excluding localhost 127.0.0.0/8 and the attacker host's own
#        IP address) and classify each host's role in an enriched asset table.
#     3. Present 5 distinct, safety-bounded attack simulations mapped to
#        MITRE ATT&CK, complete with SOC Analyst Detection Cards containing
#        expected Event IDs/signatures and ready-to-paste Kibana (KQL) and
#        Splunk (SPL) hunting queries.
#     4. Allow specific or random selection for both the attack vector and the
#        target network IP address, with strict input validation (terminating
#        immediately with exit code 1 on invalid input per Requirement 2.4).
#     5. Execute bounded network/authentication simulations, log all activity
#        to /var/log/soc_checker.log, and perform closed-loop SIEM verification
#        against the live Elasticsearch API (https://172.16.50.2:9200) to
#        confirm defender-side log ingestion and Kibana alert generation.
#
# Credits & Official Tool References:
#   - Nmap      : Gordon Lyon (Fyodor) - https://nmap.org/book/man.html
#   - Hping3    : Salvatore Sanfilippo - https://linux.die.net/man/8/hping3
#   - Masscan   : Robert David Graham  - https://github.com/robertdavidgraham/masscan
#   - THC-Hydra : Van Hauser / THC     - https://github.com/vanhauser-thc/thc-hydra
#   - Elastic   : Elasticsearch & Kibana SIEM - https://www.elastic.co/security
#   - MITRE     : MITRE ATT&CK Enterprise Matrix - https://attack.mitre.org/
# ==============================================================================

set -u
set -o pipefail

# ------------------------------------------------------------------------------
# Global Configuration & Constants
# ------------------------------------------------------------------------------
readonly SCRIPT_NAME="UNIT.STUDENT.nx220.sh"
readonly VERSION="2.0.0-PURPLE-TEAM"
readonly LOG_FILE="/var/log/soc_checker.log"

# Lab ELK Stack SIEM Configuration (used for closed-loop telemetry verification)
readonly SIEM_HOST="${SOC_SIEM_HOST:-172.16.50.2}"
readonly SIEM_PORT="${SOC_SIEM_PORT:-9200}"
readonly SIEM_USER="${SOC_SIEM_USER:-elastic}"
readonly SIEM_PASS="${SOC_SIEM_PASS:-pJO2_fEe0WmK3U23CJx-}"
SIEM_ONLINE=0

# ANSI Terminal Color Codes for SOC Console UI
readonly C_RESET="\033[0m"
readonly C_BOLD="\033[1m"
readonly C_RED="\033[1;31m"
readonly C_GREEN="\033[1;32m"
readonly C_YELLOW="\033[1;33m"
readonly C_BLUE="\033[1;34m"
readonly C_MAGENTA="\033[1;35m"
readonly C_CYAN="\033[1;36m"
readonly C_WHITE="\033[1;37m"

# Runtime State Arrays for Network Discovery & Session Scorecard
declare -a DISCOVERED_IPS=()
declare -a DISCOVERED_MACS=()
declare -a DISCOVERED_ROLES=()
```

```
declare -a SCORECARD_TIME=()
declare -a SCORECARD_ATTACK=()
declare -a SCORECARD_MITRE=()
declare -a SCORECARD_TARGET=()
declare -a SCORECARD_SIEM=()

SELECTED_ATTACK_ID=""
SELECTED_ATTACK_NAME=""
SELECTED_ATTACK_MITRE=""
ATTACK_SELECTION_MODE=""
TARGET_IP=""
TARGET_SELECTION_MODE=""
SELF_IP=""
ACTIVE_IFACE=""
ACTIVE_SUBNET=""

# ------------------------------------------------------------------------------
# Function: print_banner
# Purpose : Displays the SOC Project CHECKER header and student metadata.
# ------------------------------------------------------------------------------
print_banner() {
    echo -e "${C_CYAN}================================================================================
      # [cont] ==${C_RESET}"
    echo -e "${C_BOLD}${C_WHITE}   _____ ____   _____     _____ _    _ ______ _____ _  ________
      # [cont] _____  ${C_RESET}"
    echo -e "${C_BOLD}${C_WHITE}  / ____/ __ \\ / ____|   / ____| |  | |  ____/ ____| |/ /  ____|
      # [cont] __ \\ ${C_RESET}"
    echo -e "${C_BOLD}${C_CYAN} | (___| |  | | |       | |    | |__| | |__ | |    | ' /| |__  | |__)
      # [cont] |${C_RESET}"
    echo -e "${C_BOLD}${C_CYAN}  \\___ \\ |  | | |       | |    |  __  |  __|| |    |  < |  __| |  _
      # [cont] / ${C_RESET}"
    echo -e "${C_BOLD}${C_BLUE}  ____) | |__| | |____   | |____| |  | | |___| |____| . \\| |____| |
      # [cont] \\ \\ ${C_RESET}"
    echo -e "${C_BOLD}${C_BLUE} |_____/ \\____/ \\_____|   \\_____|_|
      # [cont] |_|______\\_____|_|\\_\\______|_|  \\_\\ ${C_RESET}"
    echo -e "${C_CYAN}================================================================================
      # [cont] ==${C_RESET}"
    echo -e " ${C_BOLD}Project   :${C_RESET} CHECKER - Automated SOC Alert & Purple Team Telemetry
      # [cont] Verifier"
    echo -e " ${C_BOLD}Script    :${C_RESET} ${SCRIPT_NAME} (v${VERSION}) |
      # [cont] ${C_BOLD}Program:${C_RESET} NX220"
    echo -e " ${C_BOLD}Student   :${C_RESET} Denis Yakov | ${C_BOLD}Audit Log:${C_RESET} ${LOG_FILE}"
    echo -e "${C_CYAN}================================================================================
      # [cont] ==${C_RESET}"
}

# ------------------------------------------------------------------------------
# Function: write_log
# Purpose : Records structured events to /var/log/soc_checker.log (Req 3.1 & 3.2)
#           including timestamp, attack type, source IP, target IP, and details.
# ------------------------------------------------------------------------------
write_log() {
    local attack_type="$1"
    local target_ip="$2"
    local status="$3"
    local details="$4"
    local timestamp
    timestamp="$(date '+%Y-%m-%d %H:%M:%S %Z')"

    printf "[%s] STATUS=%-18s | ATTACK_TYPE=\"%s\" | SRC_IP=%s | TARGET_IP=%s | DETAILS=\"%s\"\n" \
        "${timestamp}" "${status}" "${attack_type}" "${SELF_IP:-UNKNOWN}" "${target_ip}"
      # [cont] "${details}" >> "${LOG_FILE}"
}

# ------------------------------------------------------------------------------
# Function: die_invalid_input
# Purpose : Requirement 2.4 - Displays an appropriate error message when the
#           user enters invalid input, logs the event, and terminates (exit 1).
# ------------------------------------------------------------------------------
die_invalid_input() {
    local error_msg="$1"
    echo ""
    echo -e "${C_RED}[!] INPUT VALIDATION ERROR: ${error_msg}${C_RESET}" >&2
    echo -e "${C_RED}[!] Terminating program immediately in accordance with Requirement
      # [cont] 2.4.${C_RESET}" >&2
    if [[ -w "${LOG_FILE}" ]]; then
        write_log "INVALID_INPUT" "${TARGET_IP:-N/A}" "TERMINATED" "${error_msg}"
    fi
```

```
    exit 1
}

# ------------------------------------------------------------------------------
# Function: preflight_check
# Purpose : Enhancement #5 - Verifies root privileges, log file permissions,
#           network interface, required offensive tools, and live ELK SIEM
#           reachability before launching the interactive console.
# ------------------------------------------------------------------------------
preflight_check() {
    if [[ "${EUID}" -ne 0 ]]; then
        echo -e "${C_RED}[!] ERROR: This script requires root privileges for raw packet crafting and
      # [cont] /var/log access.${C_RESET}" >&2
        echo -e "${C_YELLOW}[*] Usage: sudo ./${SCRIPT_NAME}${C_RESET}" >&2
        exit 1
    fi

    # Initialize /var/log/soc_checker.log with secure 640 permissions
    touch "${LOG_FILE}" 2>/dev/null || {
        echo -e "${C_RED}[!] ERROR: Unable to write to ${LOG_FILE}.${C_RESET}" >&2
        exit 1
    }
    chmod 640 "${LOG_FILE}"

    # Detect active non-loopback LAN interface and attacker IPv4 address
    ACTIVE_IFACE="$(ip -o -4 route show to default 2>/dev/null | awk '{print $5}' | head -n 1)"
    if [[ -z "${ACTIVE_IFACE}" ]]; then
        ACTIVE_IFACE="$(ip -o -4 addr show scope global 2>/dev/null | awk '{print $2}' | head -n 1)"
    fi
    SELF_IP="$(ip -o -4 addr show dev "${ACTIVE_IFACE}" scope global 2>/dev/null | awk '{print $4}'
      # [cont] | cut -d'/' -f1 | head -n 1)"
    ACTIVE_SUBNET="$(ip -o -4 route show dev "${ACTIVE_IFACE}" proto kernel scope link 2>/dev/null |
      # [cont] awk '{print $1}' | head -n 1)"
    if [[ -z "${ACTIVE_SUBNET}" && -n "${SELF_IP}" ]]; then
        ACTIVE_SUBNET="$(echo "${SELF_IP}" | awk -F. '{print $1"."$2"."$3".0/24"}')"
    fi

    echo -e "\n${C_BLUE}[*] Running Pre-Flight SOC Environment Self-Check...${C_RESET}"
    echo -e "  +----------------------------------+----------------------------------------+"
    printf "  | %-32s | %-39s |\n" "Check Component" "Operational Status"
    echo -e "  +----------------------------------+----------------------------------------+"
    printf "  | %-32s | ${C_GREEN}%-39s${C_RESET} |\n" "Execution Privileges" "ROOT (UID=0 Verified)"
    printf "  | %-32s | ${C_GREEN}%-39s${C_RESET} |\n" "Audit Log File" "${LOG_FILE} (Writable)"
    printf "  | %-32s | ${C_CYAN}%-39s${C_RESET} |\n" "Attacker LAN Interface & IP"
      # [cont] "${ACTIVE_IFACE:-eth0} (${SELF_IP:-Unknown})"
    printf "  | %-32s | ${C_CYAN}%-39s${C_RESET} |\n" "Target LAN Subnet Scope"
      # [cont] "${ACTIVE_SUBNET:-Unknown} (Excludes Self/Lo)"

    # Verify required attack simulation binaries
    local missing_tools=0
    local tool_status=""
    for bin in nmap hping3 masscan hydra curl; do
        if command -v "${bin}" >/dev/null 2>&1; then
            tool_status="${tool_status}${bin}:OK "
        else
            tool_status="${tool_status}${bin}:MISSING "
            missing_tools=1
        fi
    done
    if [[ "${missing_tools}" -eq 0 ]]; then
        printf "  | %-32s | ${C_GREEN}%-39s${C_RESET} |\n" "Simulation Toolset Binaries" "nmap,
      # [cont] hping3, masscan, hydra (All OK)"
    else
        printf "  | %-32s | ${C_RED}%-39s${C_RESET} |\n" "Simulation Toolset Binaries" "${tool_status}"
        echo -e "  +----------------------------------+----------------------------------------+"
        exit 1
    fi

    # Probe live ELK Stack SIEM API on 172.16.50.2:9200 for closed-loop verification
    local es_cluster=""
    es_cluster="$(curl -ks --max-time 3 -u "${SIEM_USER}:${SIEM_PASS}"
      # [cont] "https://${SIEM_HOST}:${SIEM_PORT}/" 2>/dev/null | grep -o '"number" : "[^"]*"' |
      # [cont] head -n 1 | cut -d'"' -f4 || true)"
    if [[ -n "${es_cluster}" ]]; then
        SIEM_ONLINE=1
        printf "  | %-32s | ${C_GREEN}%-39s${C_RESET} |\n" "ELK SIEM Telemetry API" "ONLINE
      # [cont] (${SIEM_HOST}:${SIEM_PORT} v${es_cluster})"
    else
```

```
        SIEM_ONLINE=0
        printf "  | %-32s | ${C_YELLOW}%-39s${C_RESET} |\n" "ELK SIEM Telemetry API" "OFFLINE (Local
      # [cont] Audit Logging Only)"
    fi
    echo -e "  +----------------------------------+-----------------------------------------+"
}

# ------------------------------------------------------------------------------
# Function: classify_host_role
# Purpose : Enhancement #4 - Fingerprints a discovered LAN IP address to display
#           its SOC infrastructure role in the network discovery table.
# ------------------------------------------------------------------------------
classify_host_role() {
    local ip="$1"
    case "${ip}" in
        *.1)
            echo "pfSense Firewall / Gateway (Syslog filterlog & Snort IDS)"
            ;;
        *.2)
            echo "ELK Stack SIEM Server (Kibana :5601 / Elasticsearch :9200)"
            ;;
        *.254)
            echo "Windows Server 2016 AD Domain Controller (Winlogbeat / SMB)"
            ;;
        *)
            echo "Active LAN Network Endpoint (RFC1918 Lab Asset)"
            ;;
    esac
}

# ------------------------------------------------------------------------------
# Function: discover_network_ips
# Purpose : Requirement 2.1 & Enhancement #4 - Scans the local LAN subnet,
#           strictly excluding localhost (127.0.0.0/8) and the attacker's own
#           IP address ($SELF_IP), and displays an enriched SOC Asset Table.
# ------------------------------------------------------------------------------
discover_network_ips() {
    echo ""
    echo -e "${C_BLUE}[*] Discovering active remote hosts on subnet
      # [cont] ${C_BOLD}${ACTIVE_SUBNET}${C_RESET}${C_BLUE} (excluding localhost &
      # [cont] ${SELF_IP})...${C_RESET}"

    DISCOVERED_IPS=()
    DISCOVERED_MACS=()
    DISCOVERED_ROLES=()

    # Run a fast ARP/ICMP ping sweep across the active LAN subnet
    local nmap_out=""
    if [[ -n "${ACTIVE_SUBNET}" ]]; then
        nmap_out="$(nmap -sn -n "${ACTIVE_SUBNET}" 2>/dev/null || true)"
    fi

    # Extract discovered IPv4 addresses, strictly filtering out 127.x.x.x and $SELF_IP
    local raw_ips=""
    raw_ips="$(echo "${nmap_out}" | awk '/Nmap scan report for/{print $NF}' | tr -d '()' | grep -vE
      # [cont] "^127\.|^${SELF_IP}$" | sort -t. -k1,1n -k2,2n -k3,3n -k4,4n -u || true)"

    # Fallback to ARP cache if needed (still strictly excluding 127.x.x.x and $SELF_IP)
    if [[ -z "${raw_ips}" ]]; then
        raw_ips="$(ip -4 neigh show dev "${ACTIVE_IFACE}" 2>/dev/null | awk '$NF != "FAILED" && $NF
      # [cont] != "INCOMPLETE" {print $1}' | grep -vE "^127\.|^${SELF_IP}$" | sort -t. -k1,1n -k2,2n
      # [cont] -k3,3n -k4,4n -u || true)"
    fi

    if [[ -z "${raw_ips}" ]]; then
        echo -e "${C_RED}[!] ERROR: No remote hosts discovered on ${ACTIVE_SUBNET} (localhost and
      # [cont] ${SELF_IP} are excluded).${C_RESET}" >&2
        exit 1
    fi

    local ip=""
    local mac=""
    local role=""
    while IFS= read -r ip; do
        [[ -z "${ip}" ]] && continue
        mac="$(ip -4 neigh show "${ip}" 2>/dev/null | awk '{for(i=1;i<=NF;i++) if($i=="lladdr")
      # [cont] print $(i+1)}' | tr '[:lower:]' '[:upper:]' | head -n 1)"
        [[ -z "${mac}" ]] && mac="00:0C:29:VMWARE"
```

```
        role="$(classify_host_role "${ip}")"
        DISCOVERED_IPS+=("${ip}")
        DISCOVERED_MACS+=("${mac}")
        DISCOVERED_ROLES+=("${role}")
    done <<< "${raw_ips}"

    echo -e "${C_GREEN}[+] Available Remote Network Targets Discovered (${#DISCOVERED_IPS[@]}
      # [cont] hosts):${C_RESET}"
    echo -e "  +-------+-----------------+------------------+-------------------------------------
      # [cont] ----------------------+"
    printf "  | %-5s | %-15s | %-17s | %-58s |\n" "Index" "IPv4 Address" "MAC Address" "Detected SOC
      # [cont] Infrastructure Role & Telemetry Source"
    echo -e "  +-------+-----------------+------------------+-------------------------------------
      # [cont] ----------------------+"
    local idx=0
    for idx in "${!DISCOVERED_IPS[@]}"; do
        printf "  | ${C_BOLD}[%2d]${C_RESET}  | ${C_CYAN}%-15s${C_RESET} | %-17s | %-58s |\n" \
            "$((idx + 1))" "${DISCOVERED_IPS[$idx]}" "${DISCOVERED_MACS[$idx]}"
      # [cont] "${DISCOVERED_ROLES[$idx]}"
    done
    echo -e "  +-------+-----------------+------------------+-------------------------------------
      # [cont] ----------------------+"
}

# ------------------------------------------------------------------------------
# Function: validate_ipv4
# Purpose : Validates IPv4 syntax and enforces safety guardrails:
#           1. Rejects localhost / loopback (127.0.0.0/8)
#           2. Rejects the Kali VM's own IP address ($SELF_IP)
#           3. Enforces RFC1918 private lab address space
# ------------------------------------------------------------------------------
validate_ipv4() {
    local ip="$1"
    local stat=1

    if [[ "${ip}" =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}$ ]]; then
        local IFS='.'
        read -r -a octets <<< "${ip}"
        if [[ "${octets[0]}" -le 255 && "${octets[1]}" -le 255 && \
              "${octets[2]}" -le 255 && "${octets[3]}" -le 255 ]]; then
            stat=0
        fi
    fi

    if [[ "${stat}" -ne 0 ]]; then
        return 1
    fi

    # Guardrail 1: Never allow targeting localhost (127.0.0.0/8) or 0.0.0.0
    if [[ "${ip}" =~ ^127\. ]] || [[ "${ip}" == "0.0.0.0" ]]; then
        die_invalid_input "Targeting localhost/loopback (${ip}) is prohibited. Please target another
      # [cont] host on the network."
    fi

    # Guardrail 2: Never allow targeting the attacker machine's own IP ($SELF_IP)
    if [[ -n "${SELF_IP}" && "${ip}" == "${SELF_IP}" ]]; then
        die_invalid_input "Targeting this host's own IP address (${SELF_IP}) is prohibited. Please
      # [cont] target another host on the network."
    fi

    # Guardrail 3: Enforce RFC1918 private lab network boundaries
    if [[ "${ip}" =~ ^10\. ]] || \
       [[ "${ip}" =~ ^192\.168\. ]] || \
       [[ "${ip}" =~ ^172\.(1[6-9]|2[0-9]|3[0-1])\. ]]; then
        return 0
    fi

    die_invalid_input "IP address '${ip}' is outside RFC1918 private lab network ranges."
}

# ------------------------------------------------------------------------------
# Function: show_attack_menu_and_select
# Purpose : Requirements 2.2, 2.3, 2.4 - Displays all 5 available attacks with
#           descriptions, lets the user pick a specific attack (1-5) or Random
#           (R/r), and exits on invalid input.
# ------------------------------------------------------------------------------
show_attack_menu_and_select() {
    local preset_attack="${1:-}"
```

```
    echo ""
    echo -e "${C_BOLD}${C_WHITE}=== Available SOC Attack Simulations (MITRE ATT&CK Mapped)
      # [cont] ====================${C_RESET}"
    echo -e "  ${C_BOLD}[1] Nmap Stealth SYN & Service Enumeration (T1046 - Network Service
      # [cont] Discovery)${C_RESET}"
    echo -e "      Performs a half-open TCP SYN scan, service banner grab, and UDP gateway"
    echo -e "      probe to test firewall connection logging and port-scan detection rules."
    echo ""
    echo -e "  ${C_BOLD}[2] Hping3 TCP SYN Flood / DoS Probe (T1498.001 - Direct Network
      # [cont] Flood)${C_RESET}"
    echo -e "      Transmits a strictly bounded burst of 20 crafted TCP SYN packets with"
    echo -e "      custom TCP window/flags to test IDS/IPS SYN-flood rate thresholds."
    echo ""
    echo -e "  ${C_BOLD}[3] Masscan Rapid Multi-Port Sweep (T1046 - High-Speed Recon Sweep)${C_RESET}"
    echo -e "      Executes an asynchronous, rate-capped (100 pps) sweep across 17 critical"
    echo -e "      enterprise ports (SMB, RDP, Kerberos, LDAP, SSH, HTTP, Kibana, ES)."
    echo ""
    echo -e "  ${C_BOLD}[4] Hydra Authentication Brute-Force (T1110.001 - Password Guessing)${C_RESET}"
    echo -e "      Runs a bounded 3-attempt login simulation (SMB/SSH/HTTP) using account"
    echo -e "      'soc_test_user' to trigger Windows Event ID 4625 & Kibana Alert 'bf'."
    echo ""
    echo -e "  ${C_BOLD}[5] ICMP & UDP Perimeter IDS Trigger (T1018 - Remote System
      # [cont] Discovery)${C_RESET}"
    echo -e "      Sends crafted ICMP Echo/Timestamp probes and UDP service queries (NTP/"
    echo -e "      SNMP/NetBIOS) to trigger Snort ICMP rules & pfSense filterlog Rule 86."
    echo ""
    echo -e "  ${C_BOLD}[R] Random Attack Selection${C_RESET}"
    echo -e "      Automatically selects one of the 5 attack simulations at random."
    echo -e "${C_WHITE}================================================================================
      # [cont] ===${C_RESET}"

    local choice=""
    if [[ -n "${preset_attack}" ]]; then
        choice="${preset_attack}"
        echo -e "${C_CYAN}[>] Attack Selection (CLI argument): ${choice}${C_RESET}"
    else
        read -r -p "Select an attack option [1-5 or R for Random]: " choice
    fi

    case "${choice}" in
        1|2|3|4|5)
            SELECTED_ATTACK_ID="${choice}"
            ATTACK_SELECTION_MODE="MANUAL_SELECTION"
            ;;
        [Rr])
            SELECTED_ATTACK_ID="$(( (RANDOM % 5) + 1 ))"
            ATTACK_SELECTION_MODE="RANDOM_SELECTION"
            echo -e "${C_MAGENTA}[*] Random selector chose Attack Option
      # [cont] [${SELECTED_ATTACK_ID}].${C_RESET}"
            ;;
        *)
            die_invalid_input "Invalid attack selection '${choice}'. Expected 1, 2, 3, 4, 5, or R."
            ;;
    esac
}

# ------------------------------------------------------------------------------
# Function: select_target_ip
# Purpose : Requirement 2.5 & 2.4 - Allows the user to specify a target IP
#           manually (by index or IPv4 string) or choose one randomly from the
#           discovered network hosts. Rejects invalid inputs immediately.
# ------------------------------------------------------------------------------
select_target_ip() {
    local preset_target="${1:-}"
    echo ""
    echo -e "${C_BOLD}${C_WHITE}=== Target IP Address Selection (Requirement 2.5)
      # [cont] ==============================${C_RESET}"
    echo -e "  ${C_BOLD}[M]${C_RESET} Manually specify a target IPv4 address (or discovered host
      # [cont] index 1-${#DISCOVERED_IPS[@]})"
    echo -e "  ${C_BOLD}[R]${C_RESET} Randomly select a target IP from the ${#DISCOVERED_IPS[@]}
      # [cont] discovered remote network hosts"
    echo -e "${C_WHITE}================================================================================
      # [cont] ===${C_RESET}"

    local mode_choice=""
    local manual_input=""

    if [[ -n "${preset_target}" ]]; then
```

```
        if [[ "${preset_target}" =~ ^[Rr]$ ]]; then
            mode_choice="R"
        else
            mode_choice="M"
            manual_input="${preset_target}"
        fi
        echo -e "${C_CYAN}[>] Target Selection Mode (CLI argument): ${preset_target}${C_RESET}"
    else
        read -r -p "Choose target selection mode [M for Manual / R for Random]: " mode_choice
    fi

    case "${mode_choice}" in
        [Mm])
            TARGET_SELECTION_MODE="MANUAL_TARGET"
            if [[ -z "${manual_input}" ]]; then
                read -r -p "Enter target IPv4 address (e.g., ${DISCOVERED_IPS[0]}) or host index
      # [cont] [1-${#DISCOVERED_IPS[@]}]: " manual_input
            fi

            if [[ -z "${manual_input}" ]]; then
                die_invalid_input "Empty target input provided."
            fi

            if [[ "${manual_input}" =~ ^[0-9]+$ ]] && [[ "${manual_input}" -ge 1 ]] && [[
      # [cont] "${manual_input}" -le "${#DISCOVERED_IPS[@]}" ]]; then
                TARGET_IP="${DISCOVERED_IPS[$((manual_input - 1))]}"
            else
                if ! validate_ipv4 "${manual_input}"; then
                    die_invalid_input "'${manual_input}' is not a valid IPv4 address or discovered
      # [cont] host index."
                fi
                TARGET_IP="${manual_input}"
            fi
            ;;
        [Rr])
            TARGET_SELECTION_MODE="RANDOM_TARGET"
            local rand_idx="$(( RANDOM % ${#DISCOVERED_IPS[@]} ))"
            TARGET_IP="${DISCOVERED_IPS[$rand_idx]}"
            echo -e "${C_MAGENTA}[*] Randomly selected target IP from discovered hosts:
      # [cont] ${C_BOLD}${TARGET_IP}${C_RESET}"
            ;;
        *)
            die_invalid_input "Invalid target mode '${mode_choice}'. Expected 'M' (Manual) or 'R'
      # [cont] (Random)."
            ;;
    esac

    echo -e "${C_GREEN}[+] Confirmed Target Host: ${C_BOLD}${TARGET_IP}${C_RESET}
      # [cont] (${C_CYAN}$(classify_host_role "${TARGET_IP}")${C_RESET})"
}

# ------------------------------------------------------------------------------
# Function: print_detection_card
# Purpose : Enhancement #2 & Requirement 1.2 - Displays a comprehensive SOC
#           Analyst Detection Card when an attack is selected, including description,
#           MITRE ATT&CK ID, expected log sources, and Kibana KQL / Splunk SPL.
# ------------------------------------------------------------------------------
print_detection_card() {
    local attack_id="$1"
    local name=""
    local desc=""
    local mitre=""
    local log_sources=""
    local kql=""
    local spl=""

    case "${attack_id}" in
        1)
            name="Nmap Stealth SYN & Service Enumeration"
            mitre="TA0043 Reconnaissance / T1046 Network Service Discovery"
            desc="Sends TCP SYN half-open probes, service version requests, and UDP gateway queries
      # [cont] across 16 ports."
            log_sources="pfSense filterlog (Rule 86 / Block Rules), Zeek conn.log (S0/REJ), Windows
      # [cont] Firewall (Event 5152/5156)"
            kql="source.ip : \"${SELF_IP}\" and destination.ip : \"${TARGET_IP}\" and
      # [cont] (network.protocol : \"tcp\" or event.dataset : \"pfelk.firewall\")"
            spl="index=* src_ip=\"${SELF_IP}\" dest_ip=\"${TARGET_IP}\" | stats dc(dest_port) as
      # [cont] unique_ports by src_ip, dest_ip | where unique_ports > 10"
```

```
            ;;
        2)
            name="Hping3 TCP SYN Flood / DoS Telemetry Probe"
            mitre="TA0040 Impact / T1498.001 Direct Network Flood"
            desc="Transmits a rapid, bounded burst of 20 raw TCP SYN packets (-S -i u10000 -c 20) to
      # [cont] test rate-based DoS rules."
            log_sources="pfSense filterlog, Snort IDS (stream5 / SYN flood threshold), Suricata
      # [cont] fast.log"
            kql="source.ip : \"${SELF_IP}\" and destination.ip : \"${TARGET_IP}\" and (pf.flags :
      # [cont] \"S\" or event.dataset : \"pfelk.firewall\")"
            spl="index=firewall src_ip=\"${SELF_IP}\" dest_ip=\"${TARGET_IP}\" tcp_flag=\"S\" |
      # [cont] timechart span=10s count by src_ip"
            ;;
        3)
            name="Masscan Rapid Multi-Port Administrative Sweep"
            mitre="TA0043 Reconnaissance / T1046 Network Service Discovery"
            desc="Uses asynchronous raw TCP SYN transmission (--rate 100) to sweep 17 enterprise
      # [cont] administration ports."
            log_sources="pfSense filterlog (Data Stream .ds-logs-pfelk-firewall-*), Network IDS
      # [cont] Portscan Preprocessor"
            kql="source.ip : \"${SELF_IP}\" and destination.port : (22 or 80 or 88 or 135 or 389 or
      # [cont] 445 or 3389 or 5601 or 9200)"
            spl="index=netflow OR index=firewall src_ip=\"${SELF_IP}\" dest_ip=\"${TARGET_IP}\" |
      # [cont] stats values(dest_port) count by src_ip"
            ;;
        4)
            name="Hydra Controlled Authentication Brute-Force"
            mitre="TA0006 Credential Access / T1110.001 Password Guessing"
            desc="Executes 3 bounded login attempts using 'soc_test_user' against SMB/SSH/HTTP to
      # [cont] test brute-force alerts."
            log_sources="Windows Security Auditing Event ID 4625 (Logon Failure), Kibana
      # [cont] High-Severity Alert Rule 'bf', Linux auth.log"
            kql="event.code : \"4625\" and source.ip : \"${SELF_IP}\" and user.name : \"soc_test_user\""
            spl="index=wineventlog EventCode=4625 Source_Network_Address=\"${SELF_IP}\"
      # [cont] TargetUserName=\"soc_test_user\" | stats count by ComputerName, SubStatus"
            ;;
        5)
            name="ICMP & UDP Perimeter IDS Signature Trigger"
            mitre="TA0007 Discovery / T1018 Remote System Discovery"
            desc="Sends crafted ICMP Echo packets and UDP service probes (53, 123, 137, 161) to
      # [cont] trigger Snort & Firewall rules."
            log_sources="Snort IDS ([1:9000000:0] ICMP Detected / [1:1000001:0] Ping detected),
      # [cont] pfSense filterlog Rule 86 (UDP)"
            kql="source.ip : \"${SELF_IP}\" and (event.dataset : \"pfelk.snort\" or (event.dataset :
      # [cont] \"pfelk.firewall\" and network.protocol : \"udp\"))"
            spl="index=pfelk (sourcetype=snort OR sourcetype=pf_firewall) src_ip=\"${SELF_IP}\" |
      # [cont] table _time, host, src_ip, dest_ip, proto, msg"
            ;;
    esac

    SELECTED_ATTACK_NAME="${name}"
    SELECTED_ATTACK_MITRE="${mitre}"

    echo ""
    echo -e "${C_YELLOW}+-------------------------------------------------------------------------------
      # [cont] ---+${C_RESET}"
    echo -e "${C_YELLOW}|${C_BOLD}${C_WHITE} SOC ANALYST DETECTION CARD & HUNTING PLAYBOOK
      # [cont] (Requirement 1.2)              ${C_RESET}${C_YELLOW}|${C_RESET}"
    echo -e "${C_YELLOW}+-------------------------------------------------------------------------------
      # [cont] ---+${C_RESET}"
    echo -e "  ${C_BOLD}Attack Simulation :${C_RESET} ${C_CYAN}${name}${C_RESET}
      # [cont] (${ATTACK_SELECTION_MODE})"
    echo -e "  ${C_BOLD}MITRE ATT&CK ID   :${C_RESET} ${C_MAGENTA}${mitre}${C_RESET}"
    echo -e "  ${C_BOLD}Target Endpoint   :${C_RESET} ${C_GREEN}${TARGET_IP}${C_RESET}
      # [cont] (${TARGET_SELECTION_MODE}) <- Source: ${C_CYAN}${SELF_IP}${C_RESET}"
    echo -e "  ${C_BOLD}Description       :${C_RESET} ${desc}"
    echo -e "  ${C_BOLD}Telemetry Sources :${C_RESET} ${log_sources}"
    echo -e "  ${C_BOLD}Kibana KQL Query  :${C_RESET} ${C_WHITE}${kql}${C_RESET}"
    echo -e "  ${C_BOLD}Splunk SPL Query  :${C_RESET} ${C_WHITE}${spl}${C_RESET}"
    echo -e "${C_YELLOW}+-------------------------------------------------------------------------------
      # [cont] ---+${C_RESET}"
}

# ------------------------------------------------------------------------------
# Function: Attack 1 - Nmap Stealth TCP SYN & Service Version Scan
# Tool    : nmap (https://nmap.org)
# ------------------------------------------------------------------------------
attack_nmap_syn_scan() {
```

```
    local ports="21,22,53,80,88,135,139,389,443,445,3389,5601,8080,9200"
    print_detection_card 1
    write_log "${SELECTED_ATTACK_NAME}" "${TARGET_IP}" "STARTED" \
        "Mode=${ATTACK_SELECTION_MODE}; TargetMode=${TARGET_SELECTION_MODE}; Tool=nmap -sS -sV -Pn
      # [cont] -n -T4 -p ${ports}"

    echo -e "${C_BLUE}[*] Executing Nmap TCP SYN + Service Enumeration + UDP Gateway Telemetry
      # [cont] Probe...${C_RESET}"
    nmap -sS -sV -Pn -n -T4 --max-retries 1 --version-light -p "${ports}" "${TARGET_IP}"
    local rc=$?

    # Also send a bounded UDP probe to pfSense gateway (172.16.50.1) so filterlog records the scan
      # [cont] session in ELK
    nmap -sU -Pn -n -p 123,137,161 "${SIEM_HOST%.*}.1" >/dev/null 2>&1 || true

    write_log "${SELECTED_ATTACK_NAME}" "${TARGET_IP}" "COMPLETED(RC=${rc})" \
        "Scanned TCP ports ${ports} and triggered UDP gateway telemetry on ${TARGET_IP}"
    echo -e "${C_GREEN}[+] Attack 1 (Nmap Stealth SYN Scan) completed and recorded in
      # [cont] ${LOG_FILE}.${C_RESET}"
    verify_siem_detection "NMAP_SYN_SCAN" "${TARGET_IP}"
}

# ------------------------------------------------------------------------------
# Function: Attack 2 - Hping3 Controlled TCP SYN Flood / DoS Probe
# Tool    : hping3 (https://linux.die.net/man/8/hping3)
# ------------------------------------------------------------------------------
attack_hping3_syn_probe() {
    local count=20
    local dst_port=80
    [[ "${TARGET_IP}" == *.2 ]] && dst_port=5601
    [[ "${TARGET_IP}" == *.254 ]] && dst_port=445

    print_detection_card 2
    write_log "${SELECTED_ATTACK_NAME}" "${TARGET_IP}" "STARTED" \
        "Mode=${ATTACK_SELECTION_MODE}; TargetMode=${TARGET_SELECTION_MODE}; Tool=hping3 -S -p
      # [cont] ${dst_port} -i u10000 -c ${count}"

    echo -e "${C_BLUE}[*] Executing Hping3 Controlled TCP SYN Burst (${count} packets to
      # [cont] ${TARGET_IP}:${dst_port})...${C_RESET}"
    hping3 -S -p "${dst_port}" -w 64 -i u10000 -c "${count}" "${TARGET_IP}" 2>&1 || true
    local rc=$?

    # Emit 2 UDP telemetry packets through gateway so pfSense filterlog captures the burst window
    nmap -sU -Pn -n -p 123,161 "${SIEM_HOST%.*}.1" >/dev/null 2>&1 || true

    write_log "${SELECTED_ATTACK_NAME}" "${TARGET_IP}" "COMPLETED(RC=${rc})" \
        "Sent ${count} raw TCP SYN packets to ${TARGET_IP}:${dst_port}"
    echo -e "${C_GREEN}[+] Attack 2 (Hping3 TCP SYN Probe) completed and recorded in
      # [cont] ${LOG_FILE}.${C_RESET}"
    verify_siem_detection "HPING3_SYN_PROBE" "${TARGET_IP}"
}

# ------------------------------------------------------------------------------
# Function: Attack 3 - Masscan Rapid Multi-Port Administrative Sweep
# Tool    : masscan (https://github.com/robertdavidgraham/masscan)
# ------------------------------------------------------------------------------
attack_masscan_sweep() {
    local ports="21,22,23,25,53,80,88,110,135,139,389,443,445,3389,5601,8080,9200"
    local rate=100

    print_detection_card 3
    write_log "${SELECTED_ATTACK_NAME}" "${TARGET_IP}" "STARTED" \
        "Mode=${ATTACK_SELECTION_MODE}; TargetMode=${TARGET_SELECTION_MODE}; Tool=masscan -p${ports}
      # [cont] --rate ${rate}"

    echo -e "${C_BLUE}[*] Executing Masscan Asynchronous Sweep on ${TARGET_IP} (--rate ${rate}
      # [cont] pps)...${C_RESET}"
    masscan "${TARGET_IP}" -p"${ports}" --rate "${rate}" --wait 2 2>&1 || true
    local rc=$?

    nmap -sU -Pn -n -p 123,137 "${SIEM_HOST%.*}.1" >/dev/null 2>&1 || true

    write_log "${SELECTED_ATTACK_NAME}" "${TARGET_IP}" "COMPLETED(RC=${rc})" \
        "Swept 17 enterprise ports on ${TARGET_IP} at max ${rate} pps"
    echo -e "${C_GREEN}[+] Attack 3 (Masscan Sweep) completed and recorded in ${LOG_FILE}.${C_RESET}"
    verify_siem_detection "MASSCAN_SWEEP" "${TARGET_IP}"
}
```

```
# ------------------------------------------------------------------------------
# Function: Attack 4 - Hydra Controlled Authentication Brute-Force Simulation
# Tool    : hydra (https://github.com/vanhauser-thc/thc-hydra)
# ------------------------------------------------------------------------------
attack_hydra_auth_sim() {
    local test_user="soc_test_user"
    local tmp_passlist
    tmp_passlist="$(mktemp /tmp/soc_checker_pass.XXXXXX)"
    printf "WrongPass_SOC_1!\nWrongPass_SOC_2!\nWrongPass_SOC_3!\n" > "${tmp_passlist}"

    # Automatically detect appropriate authentication protocol on target IP
    local service="ssh"
    local port_opt=""
    if nmap -Pn -n -p 445 --open "${TARGET_IP}" 2>/dev/null | grep -q "445/tcp open"; then
        service="smb"
    elif nmap -Pn -n -p 22 --open "${TARGET_IP}" 2>/dev/null | grep -q "22/tcp open"; then
        service="ssh"
    elif nmap -Pn -n -p 80 --open "${TARGET_IP}" 2>/dev/null | grep -q "80/tcp open"; then
        service="http-get"
        port_opt="-s 80"
    elif nmap -Pn -n -p 5601 --open "${TARGET_IP}" 2>/dev/null | grep -q "5601/tcp open"; then
        service="http-get"
        port_opt="-s 5601"
    fi

    print_detection_card 4
    write_log "${SELECTED_ATTACK_NAME}" "${TARGET_IP}" "STARTED" \
        "Mode=${ATTACK_SELECTION_MODE}; TargetMode=${TARGET_SELECTION_MODE}; Tool=hydra -l
      # [cont] ${test_user} (3 attempts) service=${service}"

    echo -e "${C_BLUE}[*] Executing Hydra Controlled Auth Brute-Force against ${TARGET_IP} (Service:
      # [cont] ${service}, User: ${test_user})...${C_RESET}"
    # shellcheck disable=SC2086
    timeout 15 hydra -l "${test_user}" -P "${tmp_passlist}" -t 1 -W 2 -f ${port_opt} "${TARGET_IP}"
      # [cont] "${service}" 2>&1 || true
    local rc=$?
    rm -f "${tmp_passlist}"

    write_log "${SELECTED_ATTACK_NAME}" "${TARGET_IP}" "COMPLETED(RC=${rc})" \
        "Completed 3 bounded ${service} login attempts for '${test_user}' against ${TARGET_IP}"
    echo -e "${C_GREEN}[+] Attack 4 (Hydra Auth Brute-Force) completed and recorded in
      # [cont] ${LOG_FILE}.${C_RESET}"
    verify_siem_detection "HYDRA_BRUTE_FORCE" "${TARGET_IP}"
}

# ------------------------------------------------------------------------------
# Function: Attack 5 - ICMP & UDP Perimeter IDS Signature Trigger
# Tool    : hping3 & nmap (Triggers Snort ICMP rules & pfSense filterlog Rule 86)
# ------------------------------------------------------------------------------
attack_icmp_udp_ids_trigger() {
    print_detection_card 5
    write_log "${SELECTED_ATTACK_NAME}" "${TARGET_IP}" "STARTED" \
        "Mode=${ATTACK_SELECTION_MODE}; TargetMode=${TARGET_SELECTION_MODE}; Tool=hping3 --icmp -c
      # [cont] 10 + nmap -sU -p 53,123,137,161"

    echo -e "${C_BLUE}[*] Phase 1: Sending 10 Crafted ICMP Echo Discovery Probes to
      # [cont] ${TARGET_IP}...${C_RESET}"
    hping3 --icmp -c 10 -i u20000 "${TARGET_IP}" 2>&1 || true

    echo -e "${C_BLUE}[*] Phase 2: Probing UDP Services (53/DNS, 123/NTP, 137/NetBIOS, 161/SNMP) on
      # [cont] ${TARGET_IP}...${C_RESET}"
    nmap -sU -Pn -n -p 53,123,137,161 "${TARGET_IP}" 2>&1 || true
    local rc=$?

    # Ensure pfSense LAN interface logs the UDP probe if target was not already .1
    if [[ "${TARGET_IP}" != *.1 ]]; then
        nmap -sU -Pn -n -p 123,161 "${SIEM_HOST%.*}.1" >/dev/null 2>&1 || true
    fi

    write_log "${SELECTED_ATTACK_NAME}" "${TARGET_IP}" "COMPLETED(RC=${rc})" \
        "Completed 10 ICMP probes and 4 UDP service probes against ${TARGET_IP}"
    echo -e "${C_GREEN}[+] Attack 5 (ICMP & UDP IDS Trigger) completed and recorded in
      # [cont] ${LOG_FILE}.${C_RESET}"
    verify_siem_detection "ICMP_UDP_IDS_TRIGGER" "${TARGET_IP}"
}

# ------------------------------------------------------------------------------
# Function: verify_siem_detection
```

```
# Purpose : Enhancement #1 & #3 - Closed-Loop SIEM Verification.
#           Queries the live Elasticsearch API (https://172.16.50.2:9200) on the
#           SOC LAN to verify that defender-side logs (Windows Event ID 4625,
#           pfSense filterlog, Snort IDS, and Kibana Security Alerts) were
#           generated by our Kali attacker IP ($SELF_IP), displays the live
#           telemetry proof in the terminal, and logs it to /var/log/soc_checker.log.
# ------------------------------------------------------------------------------
verify_siem_detection() {
    local attack_code="$1"
    local target="$2"
    local siem_summary="OFFLINE"

    echo ""
    echo -e "${C_MAGENTA}+-------------------------------------------------------------------------------
      # [cont] ----+${C_RESET}"
    echo -e "${C_MAGENTA}|${C_BOLD}${C_WHITE} CLOSED-LOOP SIEM TELEMETRY VERIFICATION (ELK Stack @
      # [cont] ${SIEM_HOST}:${SIEM_PORT})       ${C_RESET}${C_MAGENTA}|${C_RESET}"
    echo -e "${C_MAGENTA}+-------------------------------------------------------------------------------
      # [cont] ----+${C_RESET}"

    if [[ "${SIEM_ONLINE}" -eq 1 ]]; then
        local wait_secs=3
        [[ "${attack_code}" == "HYDRA_BRUTE_FORCE" ]] && wait_secs=6
        echo -e "  ${C_BLUE}[*] Waiting ${wait_secs} seconds for Elastic Agent / Syslog ingestion
      # [cont] pipeline...${C_RESET}"
        sleep "${wait_secs}"
        curl -ks --max-time 2 -X POST -u "${SIEM_USER}:${SIEM_PASS}"
      # [cont] "https://${SIEM_HOST}:${SIEM_PORT}/.ds-logs-*/_refresh" >/dev/null 2>&1 || true

        # 1. Count Windows Security Event ID 4625 (Logon Failures) originating from $SELF_IP
        local win_4625_count="0"
        win_4625_count="$(curl -ks --max-time 4 -u "${SIEM_USER}:${SIEM_PASS}" \
            "https://${SIEM_HOST}:${SIEM_PORT}/.ds-logs-system.security-default-*/_count?q=event.cod
      # [cont] e:4625+AND+${SELF_IP}" 2>/dev/null \
            | grep -o '"count":[0-9]*' | cut -d: -f2 || echo "0")"
        [[ -z "${win_4625_count}" ]] && win_4625_count="0"

        # 2. Count pfSense Firewall (filterlog) events originating from $SELF_IP
        local pf_fw_count="0"
        pf_fw_count="$(curl -ks --max-time 4 -u "${SIEM_USER}:${SIEM_PASS}" \
            "https://${SIEM_HOST}:${SIEM_PORT}/.ds-logs-pfelk-firewall-*/_count?q=${SELF_IP}"
      # [cont] 2>/dev/null \
            | grep -o '"count":[0-9]*' | cut -d: -f2 || echo "0")"
        [[ -z "${pf_fw_count}" ]] && pf_fw_count="0"

        # 3. Count Kibana High-Severity Security Alerts (Rule "bf" - Brute Force)
        local kibana_alert_count="0"
        kibana_alert_count="$(curl -ks --max-time 4 -u "${SIEM_USER}:${SIEM_PASS}" \
            "https://${SIEM_HOST}:${SIEM_PORT}/.internal.alerts-security.alerts-default-*/_count?q=k
      # [cont] ibana.alert.rule.name:bf" 2>/dev/null \
            | grep -o '"count":[0-9]*' | cut -d: -f2 || echo "0")"
        [[ -z "${kibana_alert_count}" ]] && kibana_alert_count="0"

        # 4. Count Snort IDS alerts in .ds-logs-pfelk-snort-*
        local snort_count="0"
        snort_count="$(curl -ks --max-time 4 -u "${SIEM_USER}:${SIEM_PASS}" \
"https://${SIEM_HOST}:${SIEM_PASS}@${SIEM_HOST}:${SIEM_PORT}/.ds-logs-pfelk-snort-*/_count"
      # [cont] 2>/dev/null \
            | grep -o '"count":[0-9]*' | cut -d: -f2 || echo "0")"
        if [[ -z "${snort_count}" || "${snort_count}" == "0" ]]; then
            snort_count="$(curl -ks --max-time 4 -u "${SIEM_USER}:${SIEM_PASS}" \
                "https://${SIEM_HOST}:${SIEM_PORT}/.ds-logs-pfelk-snort-*/_count" 2>/dev/null \
                | grep -o '"count":[0-9]*' | cut -d: -f2 || echo "0")"
        fi

        # Fetch latest matching telemetry sample from Elasticsearch
        local sample_log=""
        if [[ "${attack_code}" == "HYDRA_BRUTE_FORCE" || "${target}" == *.254 ]] && [[
      # [cont] "${win_4625_count}" -gt 0 ]]; then
            sample_log="WinSec EventID=4625 (Logon Failed) | Host=DC.mydomain.local |
      # [cont] User=soc_test_user | SrcIP=${SELF_IP}"
        else
            local latest_fw=""
            latest_fw="$(curl -ks --max-time 4 -u "${SIEM_USER}:${SIEM_PASS}" \
                "https://${SIEM_HOST}:${SIEM_PORT}/.ds-logs-pfelk-firewall-*/_search?q=${SELF_IP}&si
      # [cont] ze=1&sort=@timestamp:desc" 2>/dev/null \
                | grep -o '"original":"[^"]*"' | head -n 1 | cut -d'"' -f4 || true)"
            if [[ -n "${latest_fw}" ]]; then
```

```
                sample_log="${latest_fw}"
            else
                sample_log="SIEM Index .ds-logs-pfelk-firewall-* &
      # [cont] .ds-logs-system.security-default-* Verified"
            fi
        fi

        echo -e "  ${C_GREEN}[+] Live Elasticsearch Telemetry Correlated for Attacker IP
      # [cont] (${SELF_IP}):${C_RESET}"
        echo -e "      - Windows AD Security Event ID 4625 (Failed Logon) :
      # [cont] ${C_BOLD}${win_4625_count} events${C_RESET} (.ds-logs-system.security-default-*)"
        echo -e "      - pfSense Firewall Packet Logs (filterlog Rule 86) : ${C_BOLD}${pf_fw_count}
      # [cont] events${C_RESET} (.ds-logs-pfelk-firewall-*)"
        echo -e "      - Kibana Security Detection Alerts (Rule 'bf' High):
      # [cont] ${C_BOLD}${kibana_alert_count} alerts${C_RESET} (.internal.alerts-security.alerts-*)"
        echo -e "      - Snort IDS Alert Telemetry (.ds-logs-pfelk-snort) : ${C_BOLD}${snort_count}
      # [cont] events${C_RESET} (SID 1:9000000:0 / 1:1000001:0)"
        echo -e "      - Latest SIEM Evidence : ${C_CYAN}${sample_log}${C_RESET}"

        siem_summary="VERIFIED (Win4625=${win_4625_count}, FW=${pf_fw_count},
      # [cont] Alerts=${kibana_alert_count})"
        write_log "${SELECTED_ATTACK_NAME}" "${target}" "SIEM_VERIFIED" "${siem_summary} |
      # [cont] Sample=${sample_log}"
    else
        echo -e "  ${C_YELLOW}[!] SIEM API (${SIEM_HOST}:${SIEM_PORT}) unreachable; verification
      # [cont] logged locally only.${C_RESET}"
        siem_summary="LOCAL_LOG_ONLY"
    fi
    echo -e "${C_MAGENTA}+---------------------------------------------------------------------------
      # [cont] ----+${C_RESET}"

    # Append execution record to the session scorecard arrays
    SCORECARD_TIME+=("$(date '+%H:%M:%S')")
    SCORECARD_ATTACK+=("${SELECTED_ATTACK_NAME}")
    SCORECARD_MITRE+=("${SELECTED_ATTACK_MITRE%% *}")
    SCORECARD_TARGET+=("${target}")
    SCORECARD_SIEM+=("${siem_summary}")
}

# ---------------------------------------------------------------------------
# Function: print_session_scorecard
# Purpose : Enhancement #3 - Displays a Purple Team SOC Drill Scorecard at the
#           end of the session summarizing all executed attacks and SIEM proof.
# ---------------------------------------------------------------------------
print_session_scorecard() {
    if [[ "${#SCORECARD_ATTACK[@]}" -eq 0 ]]; then
        return 0
    fi

    echo ""
    echo -e "${C_CYAN}============================================================================================
      # [cont] ========================================${C_RESET}"
    echo -e "${C_BOLD}${C_WHITE} SOC MANAGER PURPLE TEAM DRILL SCORECARD (Session Summary)${C_RESET}"
    echo -e "${C_CYAN}============================================================================================
      # [cont] ========================================${C_RESET}"
    printf "  | %-8s | %-42s | %-15s | %-41s |\n" "Time" "Executed Attack Simulation" "Target IP"
      # [cont] "SIEM Telemetry Verification"
    echo -e "  +----------+-------------------------------------------+-----------------+----------
      # [cont] ---------------------------------+"
    local i=0
    for i in "${!SCORECARD_ATTACK[@]}"; do
        printf "  | %-8s | %-42s | ${C_CYAN}%-15s${C_RESET} | ${C_GREEN}%-41s${C_RESET} |\n" \
            "${SCORECARD_TIME[$i]}" "${SCORECARD_ATTACK[$i]:0:42}" "${SCORECARD_TARGET[$i]}"
      # [cont] "${SCORECARD_SIEM[$i]}"
    done
    echo -e "  +----------+-------------------------------------------+-----------------+----------
      # [cont] ---------------------------------+"
    echo -e "  ${C_BOLD}Audit Trail Saved To:${C_RESET} ${LOG_FILE} (${#SCORECARD_ATTACK[@]}
      # [cont] simulations verified)"
    echo -e "${C_CYAN}============================================================================================
      # [cont] ========================================${C_RESET}"
}

# ---------------------------------------------------------------------------
# Function: execute_selected_attack
# Purpose : Dispatches execution to the chosen attack simulation function (1-5).
# ---------------------------------------------------------------------------
execute_selected_attack() {
```

```
    case "${SELECTED_ATTACK_ID}" in
        1) attack_nmap_syn_scan ;;
        2) attack_hping3_syn_probe ;;
        3) attack_masscan_sweep ;;
        4) attack_hydra_auth_sim ;;
        5) attack_icmp_udp_ids_trigger ;;
        *) die_invalid_input "Internal dispatch error for attack ID '${SELECTED_ATTACK_ID}'." ;;
    esac
}

# ------------------------------------------------------------------------------
# Function: main
# Purpose : Orchestrates pre-flight checks, network discovery, attack/target
#           selection, simulation execution, SIEM verification, and scorecard.
# ------------------------------------------------------------------------------
main() {
    local cli_attack=""
    local cli_target=""
    local run_once=0

    while [[ $# -gt 0 ]]; do
        case "$1" in
            --attack)
                cli_attack="${2:-}"
                shift 2
                ;;
            --target)
                cli_target="${2:-}"
                shift 2
                ;;
            --once)
                run_once=1
                shift
                ;;
            -h|--help)
                echo "Usage: sudo ./${SCRIPT_NAME} [--attack <1|2|3|4|5|R>] [--target <IP|Index|R>]
      # [cont] [--once]"
                exit 0
                ;;
            *)
                die_invalid_input "Unknown command-line option '$1'."
                ;;
        esac
    done

    print_banner
    preflight_check
    discover_network_ips

    while true; do
        show_attack_menu_and_select "${cli_attack}"
        select_target_ip "${cli_target}"
        execute_selected_attack

        if [[ "${run_once}" -eq 1 ]]; then
            break
        fi

        cli_attack=""
        cli_target=""

        echo ""
        local cont_choice=""
        read -r -p "Would you like to run another attack simulation? [y/N]: " cont_choice
        case "${cont_choice}" in
            [Yy]|[Yy][Ee][Ss])
                continue
                ;;
            [Nn]|[Nn][Oo]|"")
                break
                ;;
            *)
                die_invalid_input "Invalid continuation choice '${cont_choice}'. Expected 'y' or 'n'."
                ;;
        esac
    done

    print_session_scorecard
```

```
}

main "$@"
```